

Guía Didáctica: Manipulación del DOM y Estructuras de Datos

DESARROLLO WEB MODERNO • MATERIAL TÉCNICO DE APOYO

Esta guía de estudio paso a paso está diseñada para dominar los dos pilares fundamentales del desarrollo frontend interactivo: la selección y modificación del árbol jerárquico de la interfaz (DOM) y la correcta gestión e iteración de flujos de datos modernos en JavaScript.

Parte 1: Selección y Manipulación de Nodos (El DOM)

El **Document Object Model (DOM)** es la interfaz de programación para documentos HTML. El navegador web transforma el código fuente estructurado en un árbol de nodos manipulables en tiempo real mediante scripts.

1. Selección de Elementos: `getElementById` VS `querySelector`

- `document.getElementById('id')` : Retorna la referencia del elemento cuyo atributo `id` coincida exactamente con la cadena de texto provista. Es el método más rápido y directo cuando se trabaja con identificadores únicos.
- `document.querySelector('selector')` : Retorna el **primer** elemento dentro del documento que coincida con el selector o grupo de selectores CSS especificados. Permite una flexibilidad absoluta (selectores por clase, etiquetas, atributos o jerarquías complejas).

```
// Selección utilizando un identificador único directo
const contenedorPrincipal = document.getElementById('main-container');

// Selección flexible utilizando sintaxis de selectores CSS (. para clases)
const primerItemLista = document.querySelector('.lista-estudiantes .item-activo');
```

2. Modificación de Contenido: `innerText` VS `innerHTML`

- `innerText` : Permite leer o modificar el contenido de texto plano observable de un nodo y sus descendientes. Si se asigna una cadena de texto que incluya etiquetas HTML, el navegador las renderizará como texto literal sin interpretarlas. Protege de inyecciones de código.
- `innerHTML` : Permite leer o reemplazar la sintaxis HTML interna de un elemento de forma directa. El navegador interpretará, parseará y renderizará cualquier etiqueta HTML válida que se incluya en el string asignado.

```
const contenedor = document.querySelector('#contenedor-dinamico');

// Asignación con innerText: Se observarán de forma literal las etiquetas en pantalla
contenedor.innerText = "<p>Texto Seguro</p>";

// Asignación con innerHTML: El navegador renderizará un nuevo párrafo estructurado
contenedor.innerHTML = "<p>Texto Interpretado por el Navegador</p>";
```

3. Creación y Eliminación de Nodos en el Árbol

Para alterar la estructura física del documento de manera dinámica, se deben instanciar elementos en la memoria volátil del script antes de ser inyectados o removidos de la jerarquía activa.

- **document.createElement('nombre_etiqueta')** : Crea un elemento HTML en memoria especificado por su nombre de etiqueta (por ejemplo, 'div', 'p' o 'label').
- **padre.appendChild(nodoHijo)** : Añade un nodo al final de la lista de hijos de un nodo padre especificado.
- **nodo.remove()** : Elimina de forma directa el elemento del árbol del DOM al que pertenece (sintaxis nativa moderna).

```
const seccionComentarios = document.querySelector('#comentarios');

// 1. Crear un elemento DIV real en memoria
const nuevoComentario = document.createElement('div');
nuevoComentario.innerText = "Este es un aporte enviado dinámicamente.";

// 2. Acoplar el elemento hijo al nodo contenedor padre
seccionComentarios.appendChild(nuevoComentario);

// 3. Eliminación directa del elemento cuando sea requerido
// nuevoComentario.remove();
```

Nota de seguridad: Cuando necesites insertar datos proporcionados directamente por el usuario (como inputs de formularios), prefiere siempre usar `innerText` o `textContent` para prevenir vulnerabilidades de Cross-Site Scripting (XSS).

Parte 2: Manejo Avanzado de Arreglos y Objetos

En las aplicaciones web modernas, los datos comúnmente se manipulan estructurados en colecciones u objetos clave-valor antes de transformarse en nodos visibles.

1. Colecciones de Datos (Estructura de Referencia)

```
const registroAcademico = [  
  { id: 101, nombre: "Marcos Torres", calificacion: 19 },  
  { id: 102, nombre: "Elena Peña", calificacion: 12 },  
  { id: 103, nombre: "Walter Gil", calificacion: 16 }  
];
```

2. Métodos de Iteración y Transformación Críticos

- **map()** : Crea un nuevo arreglo conteniendo los resultados de la llamada a una función provista sobre cada elemento. Es ideal para transformar datos puros en estructuras listas para la UI.
- **filter()** : Crea un nuevo arreglo con todos los elementos que cumplan explícitamente con la condición lógica implementada en la función de prueba.
- **forEach()** : Ejecuta la función indicada una vez por cada elemento del arreglo. No retorna una nueva colección; se utiliza para efectos secundarios estructurados.

```
// Ejemplo de MAP: Extraer solo una lista plana de nombres  
const nombresEstudiantes = registroAcademico.map(estudiante => estudiante.nombre);  
// Resultado: ["Marcos Torres", "Elena Peña", "Walter Gil"]  
  
// Ejemplo de FILTER: Filtrar estudiantes con calificación aprobatoria (>= 15)  
const alumnosAprobados = registroAcademico.filter(estudiante => estudiante.calificacion >= 15);  
  
// Ejemplo de FOREACH: Operaciones individuales directas (ej. depuración)  
registroAcademico.forEach(estudiante => {  
  console.log(`Procesando ID: ${estudiante.id}`);  
});
```

Parte 3: Intercambio de Información con Formato JSON

JSON (JavaScript Object Notation) es el formato estándar e independiente del lenguaje para el intercambio de datos estructurados entre un servidor web y una aplicación cliente.

- **JSON.stringify(valor)** : Convierte un objeto o arreglo nativo de JavaScript en una cadena de texto plana bajo la sintaxis JSON. Es indispensable para la transmisión de datos a través de peticiones HTTP.
- **JSON.parse(textoJSON)** : Toma una cadena de texto con formato JSON válido y la transforma en la estructura de datos correspondiente nativa de JavaScript (objetos o arreglos), permitiendo su manipulación inmediata.

```
const sesionDocente = { usuario: "rloreto", activo: true };

// Conversión a cadena de texto lista para transporte (Envío)
const jsonSerializado = JSON.stringify(sesionDocente);
console.log(jsonSerializado); // '{"usuario":"rloreto","activo":true}'

// Conversión de vuelta a objeto manipulable (Recepción)
const objetoParseado = JSON.parse(jsonSerializado);
console.log(objetoParseado.usuario); // "rloreto"
```

Práctica Integradora: Flujo de Datos Completo

A continuación se presenta un caso de uso integrado que simula el ciclo de vida completo de desarrollo: captura de un string JSON externo, conversión a datos manipulables, filtrado lógico e inyección estructurada y limpia en el DOM.

```
// 1. Cadena estructurada JSON simulando una respuesta de API o Base de Datos
const respuestaServidorJSON = '[{"producto":"Monitor 24","costo":150}, {"producto":"Adaptador HDMI","costo":15}, {"producto":"Impresora Láser","costo":210}]';

// 2. Reconstrucción de la estructura nativa en JavaScript
const listaInventario = JSON.parse(respuestaServidorJSON);

// 3. Selección del nodo contenedor mediante querySelector
const vitrinaProductos = document.querySelector('#lista-productos');

// 4. Filtrado lógico: Excluir ítems con costo menor o igual a $20 (Uso de filter)
const productosFiltrados = listaInventario.filter(item => item.costo > 20);

// 5. Transformación e inyección automatizada (Uso coordinado de map y appendChild)
productosFiltrados.map(item => {
  // Instanciar un nuevo nodo contenedor para la tarjeta del producto
  const tarjeta = document.createElement('div');
  tarjeta.className = 'tarjeta-producto';

  // Asignar la estructura interna de forma segura y limpia
  tarjeta.innerHTML = `
    <h3>${item.producto}</h3>
    <p>Costo de Mercado: $$${item.costo}</p>
    <hr>
  `;

  // Vincular el nuevo elemento al DOM activo
  vitrinaProductos.appendChild(tarjeta);
});
```